

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Отчет по лабораторной работе №2
«Поиск ключевых точек на изображении»

МЕТОДЫ РАСПОЗНАВАНИЯ ОБРАЗОВ

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Руководитель:

Доцент (ОИТ, ИШИТР)

Друки А.А.

Содержание

1. Цель работы	3
2. Задачи	3
3. Ход работы.....	4
4. Заключение	11

1. Цель работы

Получить навыки применения библиотеки OpenCV и алгоритма SIFT, для поиска ключевых точек на изображении.

2. Задачи

1. Загрузка каскадного классификатора и исходного изображения;
2. Обработка изображения;
3. Применение алгоритма SIFT для поиска ключевых точек и дескрипторов.

Объяснение на примере: представь, что у тебя есть фотография ландшафта с разными объектами, и ты хочешь найти уникальные и особенные точки на этой фотографии. Такие точки могут быть, например, углами зданий или интересными текстурами. Алгоритм SIFT может помочь тебе с этим.

1. Сначала алгоритм SIFT представляет изображение в виде набора размытых версий (масштабов) этого изображения. Это делается для того, чтобы отлавливать особенности на разных уровнях детализации.

2. Затем алгоритм SIFT ищет в каждом масштабе "хорошие" точки, которые отличаются от окружающих. Для этого он анализирует разницу интенсивности пикселей в разных направлениях и находит точки с высокими градиентами или краями. Например, если у нас есть угол здания, то на этом углу будет большая разница в значениях пикселей между разными направлениями.

3. Затем алгоритм SIFT ориентирует найденные точки, чтобы сделать их устойчивыми к вращению изображения. Он определяет предпочтительное направление или ориентацию каждой точки, исходя из градиентов пикселей в окрестности. Это позволяет найти особые точки, которые будут одинаковыми независимо от вращения изображения.

4. Алгоритм SIFT также создает описания для каждой найденной точки. Описание представляет собой вектор, который содержит информацию о градиентах пикселей вокруг точки. Эти описания могут быть использованы для сравнения точек на разных изображениях.

5. Когда у тебя есть найденные ключевые точки и их описания, ты можешь использовать их для различных задач, таких как сопоставление изображений, поиск объектов или даже создание панорамного изображения.

Итак, алгоритм поиска ключевых точек SIFT ищет уникальные и особенные точки на изображении с помощью размытых версий и анализа интенсивностей пикселей и краев. Затем он ориентирует эти точки и создает их описания. В результате получается набор ключевых точек, которые могут быть использованы для различных целей.

3. Ход работы

Создадим новый блокнот в Google Colab и импортируем необходимые для работы с изображениями библиотеки:

```
import cv2
```

Загрузим исходное изображение.

```
# Загрузка изображения
img = cv2.imread('/content/drive/MyDrive/Методы распознавания
образов/ben_1.jpg')
height, width, channels = img.shape

# Загрузка 2 изображения
img1 = cv2.imread('/content/drive/MyDrive/Методы распознавания
образов/ben_2.jpg')

from google.colab.patches import cv2_imshow
cv2_imshow(img)
```



Рисунок 1 – Исходное изображение

Преобразуем изображение в градации серого через среднее значение каналов и изменим размер изображения.

```
# Преобразование изображения в оттенки серого
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Изменение размера изображения
resized = cv2.resize(img, (500, 500))
cv2_imshow(resized)
```

Применим алгоритм SIFT к предобработанному изображению.

```
# Создание объекта SIFT
```

```
sift = cv2.xfeatures2d.SIFT_create()
# Нахождение ключевых точек и дескрипторов
keypoints, descriptors = sift.detectAndCompute(img, None)
keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
```

Отображение и сохранение результата:

```
# Отрисовка найденных ключевых точек на изображении
img_with_keypoints = cv2.drawKeypoints(img, keypoints, None)
cv2.imshow('img_with_keypoints')
# Сохранение результата
cv2.imwrite('image_with_keypoints.jpg', img_with_keypoints)
cv2.waitKey(0)
```



Рисунок 2 – Результат применения алгоритма SIFT

Применим FLANN matcher.

```
# Использование программы FLANN Matcher
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv2.FlannBasedMatcher(index_params, search_params)

# KNN Matcher
matches = flann.knnMatch(descriptors, descriptors1, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Построение графиков результатов сопоставления
result_img = cv2.drawMatches(img, keypoints, img1, keypoints1,
good_matches, None, flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

cv2.imshow(result_img)
```

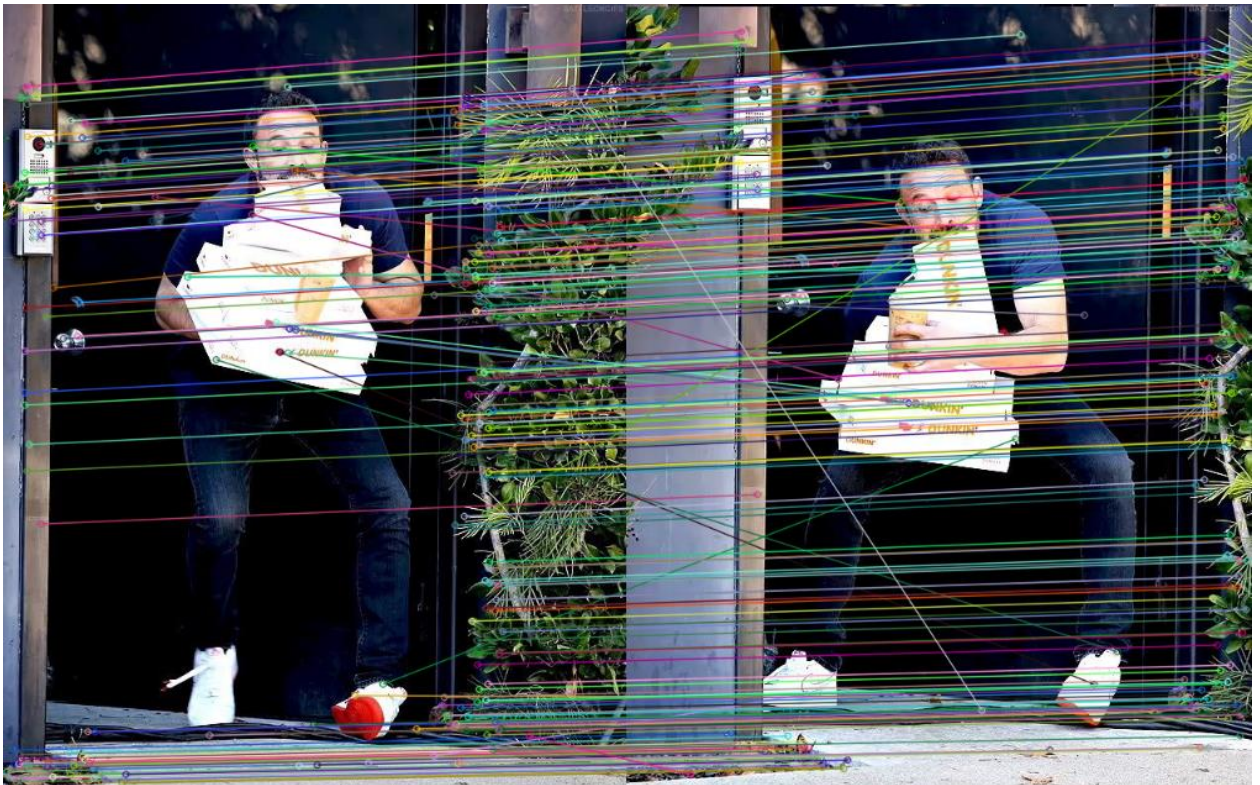


Рисунок 3 - FLANN matcher для алгоритма SIFT

Проделаем то же самое для алгоритма ORB (Oriented FAST and Rotated BRIEF):

```
orb = cv2.ORB_create()
keypoints, descriptors = orb.detectAndCompute(img, None)
keypoints1, descriptors1 = orb.detectAndCompute(img1, None)

# Отрисовка найденных ключевых точек на изображении
# img_with_keypoints = cv2.drawKeypoints(img, kp, img,
# flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
img_with_keypoints = cv2.drawKeypoints(img, keypoints, None)

cv2.imshow('img_with_keypoints')
```



Рисунок 4 – Результат применения алгоритма ORB

Применим FLANN matcher.

```
# Использование программы FLANN Matcher
FLANN_INDEX_LSH = 6
index_params= dict(algorithm = FLANN_INDEX_LSH,
                    table_number = 6, # 12
                    key_size = 12,    # 20
                    multi_probe_level = 1) #2
search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)

# KNN Matcher
matches = flann.knnMatch(descriptors, descriptors1, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Построение графиков результатов сопоставления
result_img = cv2.drawMatches(img, keypoints, img1, keypoints1,
                             good_matches, None, flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

cv2.imshow(result_img)
```



Рисунок 5 – FLANN matcher для алгоритма ORB

Проделаем то же самое для алгоритма FAST (Features from Accelerated Segment Test):

```
# FAST Features from Accelerated Segment Test
fast = cv2.FastFeatureDetector_create()
kps = fast.detect(img, None)
img_fast=cv2.drawKeypoints(img, kps, img,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imshow(img_fast)
```



Рисунок 6 – Результат применения алгоритма FAST

4. Заключение

В результате выполнения лабораторной работы были получены навыки применения библиотеки OpenCV и алгоритмов SIFT, ORB и FAST, для поиска ключевых точек на изображении.